6. Unix, Control-M and CI/CD – Interview Notes

Short brush-up notes with simple examples.

Section	Topic	Priority
A	Unix and shell scripting	High
B	Control-M	Very high
C	Git and CI/CD	High

Running example: a daily job receives a file `customer_20261104.csv`, loads it through an ETL workflow, and writes logs to `/data/logs/`.

A. Unix and Shell Scripting

1. Basic Unix Commands

Command	Purpose	Example
pwd	Show current directory	pwd
ls -ltr	List files with details, oldest first (newest at bottom)	ls -ltr /data/in
cd	Change directory	cd /data/logs
mkdir -p	Create directory (and parents)	mkdir -p /data/archive/2026
cp, mv, rm	Copy, move or rename, delete	mv a.csv /data/archive/
cat, head, tail	View file, first lines, last lines	tail -100 job.log
tail -f	Follow a log as it grows	tail -f job.log
wc -l	Count lines	wc -l customer.csv
ps -ef	List running processes	ps -ef then grep for a job name
kill	Stop a process	kill 12345 (kill -9 only as a last resort)
df -h, du -sh	Disk free space, folder size	df -h /data
echo	Print text or a variable	echo $HOME

Remember: a very common first check when a job fails is disk space (`df -h`) and file presence (`ls -l`).

2. grep

Search for text in files.

```
grep "ERROR" job.log             # lines containing ERROR
grep -i "error" job.log          # ignore case
grep -v "INFO" job.log           # exclude lines containing INFO
grep -n "ERROR" job.log          # show line numbers
grep -c "ERROR" job.log          # count matching lines
grep -r "CUSTOMER_ID" /data/sql  # search recursively in a folder
grep -E "ERROR|FATAL" job.log    # either word (extended regex)
grep -A 3 -B 2 "ERROR" job.log   # 3 lines after and 2 before the match
grep -l "ERROR" *.log            # only names of files that match
```

3. awk

Process text column by column. Default separator is whitespace; use `-F` for another one.

```
awk -F',' '{print $1, $3}' customer.csv          # print columns 1 and 3
awk -F',' '$3 > 50000 {print $0}' emp.csv        # filter rows where column 3 > 50000
awk -F',' '{sum += $3} END {print sum}' emp.csv  # total of column 3
awk 'END {print NR}' customer.csv                # number of lines
awk -F',' 'NR > 1 {print $1}' customer.csv       # skip the header line
```

Built-ins: `$0` whole line, `$1` first field, `NF` number of fields, `NR` current line number.

4. sed

Stream editor, mostly used for find and replace.

```
sed 's/Pune/Bangalore/' file.txt        # replace first match on each line
sed 's/Pune/Bangalore/g' file.txt       # replace all matches
sed -i 's/DEV/PROD/g' param.txt         # edit the file in place
sed -n '5,10p' file.txt                 # print lines 5 to 10
sed '1d' customer.csv                   # delete the first line (header)
sed '/^$/d' file.txt                    # delete blank lines
```

Tip: take a backup before `sed -i` (for example `sed -i.bak ...`).

5. find

Search for files by name, age, size or type.

```
find /data/in -name "customer_*.csv"           # by name
find /data/logs -type f -mtime +7              # files older than 7 days
find /data/in -type f -size +100M              # files bigger than 100 MB
find /data/logs -name "*.log" -mtime +30 -exec rm {} \;   # delete old logs
```

Remember: `-mtime +7` means older than 7 days, `-mtime -1` means modified in the last day.

6. sort and uniq

```
sort file.txt              # alphabetical
sort -n file.txt           # numeric
sort -r file.txt           # reverse
sort -t',' -k3 -n emp.csv  # sort a CSV by column 3 numerically
sort -u file.txt           # sort and remove duplicates
uniq -c                    # count consecutive duplicate lines
```

Important: `uniq` only removes or counts adjacent duplicates, so always `sort` first.

Example: find duplicate customer IDs in a CSV.

```
cut -d',' -f1 customer.csv | sort | uniq -d     # print IDs that appear more than once
cut -d',' -f1 customer.csv | sort | uniq -c | sort -nr | head   # top repeated IDs
```

7. cut

Extract columns or characters.

```
cut -d',' -f1,3 customer.csv    # fields 1 and 3, comma separated
cut -c1-8 file.txt              # characters 1 to 8
```

Limitation: `cut` cannot handle commas inside quoted CSV values. Use awk or a CSV tool for that.

8. chmod

Change file permissions. Permissions are read (r=4), write (w=2) and execute (x=1) for owner, group and others.

Number	Meaning	Use
755	Owner rwx, group r-x, others r-x	Scripts
644	Owner rw-, group r--, others r--	Normal files
600	Owner rw only	Private files, credentials
700	Owner rwx only	Private scripts or folders

```
chmod +x run_etl.sh      # make a script executable
chmod 755 run_etl.sh
chmod 600 password.txt
```

Common failure: "Permission denied" when a scheduler tries to run a script that is not executable or not accessible to the run-as user.

9. Shell Variables

```
LOAD_DATE=20261104                  # no spaces around =
FILE_NAME="customer_${LOAD_DATE}.csv"
echo "$FILE_NAME"
export ENV=PROD                     # make it available to child processes
TODAY=$(date +%Y%m%d)               # command substitution
```

Special variables:

Variable	Meaning
$0	Script name
$1, $2	First and second argument
$#	Number of arguments
$@	All arguments
$?	Exit code of the last command
$$	Process ID of the script

10. Loops and Conditions

```
#!/bin/bash
FILE="/data/in/customer_$(date +%Y%m%d).csv"

if [ -f "$FILE" ]; then
  echo "File found"
elif [ -d "/data/in" ]; then
  echo "Folder exists but file is missing"
else
  echo "Folder missing"
fi
```

Useful tests: `-f` file exists, `-d` directory exists, `-s` file is not empty, `-z` string is empty, `-eq -ne -gt -lt` numeric comparison, `==` and `!=` string comparison.

```
# for loop
for f in /data/in/*.csv; do
  echo "Processing $f"
done

# while loop reading a file line by line
while read -r line; do
  echo "$line"
done < list.txt

# retry loop
for i in 1 2 3; do
  ./load.sh && break
  sleep 60
done
```

11. Exit Codes

★ Very important

Every command returns an exit code. 0 means success, any non-zero value means failure. Schedulers like Control-M use the exit code to decide whether the job ended OK or Not OK.

```
./load.sh
echo $?        # prints the exit code of load.sh

if [ $? -ne 0 ]; then
  echo "Load failed"
  exit 1
fi
```

Safer script header:

```
#!/bin/bash
set -e            # stop on the first failing command
set -u            # error on undefined variables
set -o pipefail   # a failure anywhere in a pipeline fails the pipeline
```

Remember: a script's exit code is the exit code of its last command, unless you call `exit n`. A script that prints "ERROR" but exits 0 will show as successful in the scheduler.

12. Log Analysis

Typical interview scenario: "A job failed. How do you analyze the logs?"

```
tail -200 job.log                                  # most recent output
grep -n "ERROR" job.log | head                     # first errors
grep -B 5 -A 5 "ORA-" job.log                      # context around an error
grep -c "ERROR" job.log                            # how many errors
grep "ERROR" job.log | awk '{print $1}' | cut -c1-2 | sort | uniq -c   # errors per hour (if the line sta
zgrep "ERROR" job.log.gz                           # search a compressed log
ls -ltr /data/logs | tail                          # latest log files
```

Approach: find the first error (not just the last), read the lines around it, check timestamps, then identify whether it is data, connectivity, permission, space or code related.

13. File Manipulation

```
head -1 customer.csv > header.txt          # save the header
tail -n +2 customer.csv > data.csv         # remove the header (start from line 2)
wc -l customer.csv                         # line count (for reconciliation)
split -l 1000000 big.csv part_             # split into 1M-line files
cat part_* > merged.csv                    # merge files
diff old.csv new.csv                       # compare two files
tr -d '\r' < win.csv > unix.csv            # remove Windows carriage returns
gzip file.csv                              # compress
tar -czf archive.tar.gz /data/in           # create a compressed archive
sort a.txt > a_sorted.txt                  # then use comm or diff to compare sets
```

Typical checks before loading a file: file exists, not empty, record count matches the control or trailer record, header is correct, no carriage-return problems.

B. Control-M

1. Control-M Architecture

Control-M is an enterprise workload automation and job scheduling tool from BMC.

```
Control-M Client (GUI / web)  -->  Control-M/Enterprise Manager  -->  Control-M/Server  -->  Control-M/Ag
```

Component	Role
Control-M/Enterprise Manager (EM)	Central management: lets you define, view and monitor jobs from one place
Control-M/Server	The scheduling engine and database: decides which jobs run and when
Control-M/Agent	Installed on the host where jobs run; receives a job from the server, runs it and reports back the status
Client or web interface	Where you define, order, monitor and rerun jobs

Simple way to say it: the client defines the job, the server schedules it, the agent executes it.

Interview answer:

"Control-M has a central server that schedules and monitors jobs, and agents installed on target hosts that execute them. We define jobs and dependencies in the client, the server orders them daily and submits them when their conditions are met, and the agent runs the script or command and sends back the status and output."

2. Job Scheduling

A job definition normally contains:

- Job name and the folder it belongs to (folders are used to group jobs, for example per application or per batch).
- Application and sub-application (logical grouping for monitoring).
- Job type: command, script, file watcher, database, or an integration job (for example Informatica).
- Host or host group where it runs, and the run-as user.
- Command or script path with parameters, for example `/apps/etl/run_customer_load.sh 20261104`.
- Scheduling: which days (calendar), time from and until, and cycle (run every N minutes).
- Conditions, resources, and On/Do actions.

How daily scheduling works: at the start of each Control-M "day" (the New Day process), the jobs scheduled for that day are ordered. Ordered jobs then wait until their time and conditions are satisfied, and then run.

Example: a job runs every working day after 1:00 AM, only after the source file arrives.

3. Job Dependencies

Jobs depend on each other so that they run in the right order.

```
Job A (extract)  -->  Job B (load)  -->  Job C (report)
```

Ways to define dependencies:

- Conditions (most common): Job A sets an out-condition when it ends OK, and Job B waits for that as an in-condition.
- Inside a folder or Smart Folder: jobs linked in a flow.
- File watcher jobs: wait until a file appears.
- Resources: limit parallel jobs, for example only 2 jobs can use the DB connection at the same time.

4. Conditions

Type	Meaning
In-condition	A job will not start until this condition exists
Out-condition (ADD)	Condition created when the job ends successfully
Out-condition (DELETE)	Condition removed after the job ends (used to clean up)

Example:

```
Job EXTRACT_CUSTOMER:  Out-condition ADD   EXTRACT_CUSTOMER-ENDED-OK
Job LOAD_CUSTOMER:     In-condition       EXTRACT_CUSTOMER-ENDED-OK
```

Important: conditions have a date reference (for example ODAT, the original scheduling date), so today's job does not run based on yesterday's condition. Stale conditions (not cleaned up) are a common cause of jobs running too early.

Related ideas:

- Quantitative resources: a counter, for example max 3 parallel jobs.
- Control resources: shared or exclusive locks, for example only one job may update a table at a time.

5. Job Recovery

When a job fails, you have these options.

Action	When to use
Rerun	Fix the issue, then run the job again (it keeps the same order)
Hold, then Free	Hold a job to stop it running, free it when ready
Force OK	Mark the job as OK without running it, so dependents can proceed (only when approved and safe)
Delete or Undelete	Remove a job from today's run, or bring it back
Kill	Stop a running job
Bypass or Confirm	Skip a job or wait for manual confirmation

Automatic recovery using On/Do statements:

- On: check the job's exit code or output (for example code > 0 or the text "ORA-").
- Do: take an action such as Rerun (with a maximum number of reruns and wait interval), send a Shout (notification), set a condition, or set a variable.

Before rerunning, always check whether the job left partial data (duplicates, half-loaded files). This ties to ETL restartability and idempotency.

Interview answer:

"For a failed job I first check the output and log to find the root cause. Then I fix it, check for partial data, and rerun. I use Force OK only with approval when the job is not needed, and I make sure downstream jobs and SLAs are not impacted. For known transient errors I configure automatic rerun with a limit."

6. Calendars

Calendars define on which days a job runs.

- Regular calendars: list of specific dates (for example business days of the year).
- Rule-based calendars (RBC): rules such as "every weekday", "last working day of the month", "first Monday".
- Jobs can use a calendar for run dates and also exclude dates (for example bank holidays).

Examples:

- Daily load: every business day.
- Month-end report: last working day of the month.
- Weekly job: every Sunday.

Tip: if a job did not run on a certain day, check its calendar and scheduling rules first.

7. SLA Management

An SLA says that a business process must finish by a certain time (for example the morning reports must be ready by 7:00 AM).

How Control-M helps:

- Define a service (a group of critical jobs with a deadline) so Control-M tracks the whole flow.
- It calculates the critical path from expected run times (based on history) and predicts whether the service will finish on time.
- Alerts (Shouts) go out when a job is late, running long, or the service is at risk.
- Dashboards show the services and their status.

Typical settings: must start by, must complete by, maximum run time, late-submission and late-execution alerts.

Interview point: monitor not just failures but also delays. A job that has not failed but is running much longer than usual can still break an SLA.

8. Troubleshooting Failed Jobs

Step by step (say this in the interview):

1. Check the job status: Ended Not OK, Wait Condition, Wait Resource, Wait Host and so on.
2. Open the job output (sysout) and the log, and note the exit code.
3. Identify the cause: script error, data issue, database or connectivity problem, permission, missing file, disk space, or the agent or host being unavailable.
4. Check the job definition: command path, parameters, run-as user, host.
5. Check dependencies: are in-conditions present, and did upstream jobs finish properly?
6. Fix the root cause.
7. Check for partial data or duplicates.
8. Rerun (or Force OK with approval).
9. Validate that downstream jobs run and the target data is correct.
10. Communicate status and record the incident.

Common statuses:

Status	Usual meaning
Wait Condition	An in-condition is missing (upstream job not done or condition not added)
Wait Resource	Not enough resource units free
Wait Host	The agent or host is unavailable or busy
Wait Time	Job's start time has not been reached
Executing	Running
Ended OK	Finished successfully
Ended Not OK	Failed (non-zero exit code or On/Do rule)

Example: Control-M triggers an IICS or PowerCenter workflow through a script, the script returns exit code 1 when the workflow fails, and Control-M marks the job Ended Not OK. This is why correct exit codes in wrapper scripts matter.

C. Git and CI/CD

1. Git Fundamentals

Git is a distributed version control system. Every developer has a full copy of the repository.

```
Working directory  --git add-->  Staging area  --git commit-->  Local repo  --git push-->  Remote repo (G
```

Command	Purpose
git clone url	Copy a remote repository
git status	Show changes
git add file	Stage changes
git commit -m "message"	Save a snapshot locally
git push	Send commits to the remote
git fetch	Download remote changes without merging
git pull	Fetch and merge (fetch + merge)
git branch	List or create branches
git checkout or git switch	Move to a branch
git log --oneline	View history
git diff	Show differences
git stash	Temporarily shelve uncommitted changes
git tag	Mark a release version

Undoing changes:

- `git restore file`: discard changes in the working directory.
- `git reset`: move the branch back (rewrites local history, avoid on shared branches).
- `git revert commit`: create a new commit that undoes a previous one (safe for shared history).

2. Branching Strategies

Strategy	How it works	Good for
Git Flow	main, develop, feature, release and hotfix branches	Planned releases, versioned products
GitHub Flow	main plus short-lived feature branches and pull requests	Frequent releases, simple teams
Trunk-based	Everyone commits to main in small changes with feature flags	Mature CI/CD, very frequent deployments

Typical naming: `feature/CUST-123-add-lookup`, `bugfix/...`, `hotfix/...`, `release/1.4`.

Hotfix flow (Git Flow): branch from main, fix, merge into main and also back into develop.

3. Pull Requests

A pull request (PR) asks for your branch to be merged into another (for example feature into develop or main).

What happens in a PR:

1. Create a branch, commit and push.
2. Open the PR with a clear description and the ticket or CRQ reference.
3. Automated checks run (build, tests, code quality).
4. Reviewers comment and approve.
5. Fix feedback, then merge (merge commit, squash or rebase merge).

Good practice: small PRs, clear description, link to the ticket, at least one approval, passing checks before merge, and protected branches (no direct push to main).

4. Merge Conflicts

A conflict happens when two branches change the same lines of the same file and Git cannot decide which to keep.

Git marks the file like this:

```
<<<<<<< HEAD
city = 'Pune'
=======
city = 'Bangalore'
>>>>>>> feature/update-city
```

Resolve it:

1. Open the file and decide the correct final content (remove the markers).
2. `git add file`
3. `git commit` (for a merge) or `git rebase --continue` (for a rebase).

Prevention: pull often, keep branches short-lived, make small changes, and communicate when several people edit the same file.

5. Git Merge and Rebase

Both combine work from one branch into another.

Merge	Rebase
Combines branches and creates a merge commit	Replays your commits on top of the target branch
Keeps the real history, including branching	Produces a clean, linear history
Does not change existing commits	Rewrites commits (new commit IDs)
Safe for shared branches	Do not rebase branches others are using

```
git checkout feature
git merge main        # bring main's changes into feature (merge commit)

git checkout feature
git rebase main       # replay feature's commits on top of main
```

Golden rule: never rebase commits that have already been pushed and shared. Also know squash merge: it combines all commits of a PR into one commit on the target branch.

6. CI/CD Fundamentals

Term	Meaning
CI (Continuous Integration)	Every commit is automatically built and tested, so problems are found early
Continuous Delivery	Every successful build is ready to release, with a manual approval for production
Continuous Deployment	Every successful build is automatically deployed to production

Typical pipeline:

```
Commit -> Build -> Unit tests -> Code quality scan -> Package (artifact) -> Deploy to Dev -> Integration
       -> Deploy to Pre-prod -> Approval -> Deploy to Prod -> Smoke test -> Monitor
```

Key principles:

- Build once, deploy many times: the same artifact moves through all environments. Only the configuration (connections, parameters) changes.
- Automate repeatable steps (build, test, deploy).
- Fast feedback: fail early.
- Version everything (code, configuration and scripts).
- Have a rollback plan.
- Keep secrets out of code (use a secrets store).

7. Bamboo Deployment

Bamboo is Atlassian's CI/CD server.

Key concepts:

Concept	Meaning
Build plan	Defines how to build and test: stages, jobs and tasks
Stage and job	A plan has stages that run in sequence; each stage has jobs that can run in parallel
Task	A single action (checkout code, run a script, run tests)
Trigger	What starts a build (a commit, a schedule, or manual)
Artifact	Output of a build (a package, scripts, exported files)
Agent	The machine that runs the build jobs
Deployment project	Takes a release (a versioned set of artifacts) and deploys it to environments
Environment	A deployment target, for example Dev, Pre-prod, Prod
Release	A specific build version selected for deployment
Variables	Environment-specific or build-specific values (passwords are masked)

Flow:

```
Git commit  -->  Bamboo build plan (build + test)  -->  Artifact  -->  Release  -->  Deploy to Dev  -->
```

What to check after a deployment: the deployment result is successful, the deployed release version is the approved one, the correct environment was used, and post-deployment checks

(smoke tests, job runs) passed. Bamboo also has a REST API to read build and deployment status, useful for automation and validation.

8. Development-to-Production Promotion

For an ETL or data project (for example IICS, scripts and BigQuery objects), a controlled promotion looks like this.

1. Develop on a feature branch in Dev and run unit tests.
2. Raise a pull request, get a review and the checks to pass.
3. Merge to the main or release branch. The CI build creates a versioned artifact.
4. Deploy the artifact to Dev, then run integration tests.
5. Promote the same release to Pre-prod (with Pre-prod connections and parameters). Run regression and data validation, for example count reconciliation.
6. Get approvals and a change request (CRQ), and confirm the deployment window.
7. Deploy the same release to Production.
8. Run smoke tests, and monitor the first scheduled run (including the Control-M jobs).
9. If it fails, roll back to the previous release and investigate.

Things that must differ per environment (never hardcode): connections, credentials, hostnames, file paths, schemas and parameter values.

Interview answer:

"We build the artifact once and promote the same version from Dev to Pre-prod to Production, only changing environment-specific configuration. Each stage has automated tests and validation, production needs an approved change request and a deployment window, and we monitor the first run after release. We keep a rollback plan so we can redeploy the previous version quickly."

D. Quick Revision

Topic	Remember
grep	Search text (-i, -v, -n, -c, -r, -E, -A, -B)
awk	Column processing (-F, $1, NR, NF, sums)
sed	Find and replace (s/old/new/g, -i for in place)
find	Search files (-name, -mtime, -size, -exec)
sort and uniq	Always sort before uniq
chmod	r=4, w=2, x=1, so 755 for scripts, 644 for files
Exit code	0 is success, non-zero is failure, $? holds it
set -e / pipefail	Stop on error, catch pipeline failures
Control-M	Server schedules, agent executes, client defines
In and out conditions	Job waits for in-condition, adds out-condition when done
Recovery	Rerun, Force OK (with approval), On/Do automatic rerun
SLA	Monitor lateness and critical path, not only failures
Merge vs rebase	Merge keeps history, rebase makes it linear, never rebase shared branches
Pull vs fetch	Pull is fetch plus merge
Revert vs reset	Revert adds an undo commit, reset moves history back
CI vs CD	CI builds and tests every commit, CD delivers or deploys automatically
Core principle	Build once, deploy many, configuration per environment

Self-test:

1. How do you find the top 5 duplicate IDs in a CSV file?
2. Why does a script exit with code 0 even though it printed ERROR?
3. A Control-M job is in "Wait Condition". What do you check?
4. How do you safely rerun a failed load job?
5. What is the difference between git merge and git rebase?
6. How is a release promoted from Dev to Production?